EXPERIMENT 15

Career Pathways and Future of Low-Code Development

Name: *Kanishka S* | ***Register Number:*** *240701227*

Subject: *Low-Code Application Development using Zoho Creator*
Project: *Smart Campus Sustainability & Carbon Tracker*

Aim

To study the career pathways in low-code development, to identify the skills required for different roles, and to understand the future growth and evolution of low-code platforms such as Zoho Creator, with reference to the Smart Campus Sustainability & Carbon Tracker project.

Introduction to Low-Code Development

Low-code development is an approach to building software applications mainly through visual tools and configuration instead of writing large amounts of traditional program code. Applications are assembled using visual development environments, predefined components and templates. Typical building blocks include forms for capturing data, reports and dashboards for viewing it, and workflows that define how information moves through a business process. Zoho Creator is an example of such a platform, in which applications are built from forms, reports and workflows and can be extended with Deluge scripting.

In the Smart Campus Sustainability & Carbon Tracker project, for example, forms capture energy, water and waste readings, a workflow calculates the estimated carbon emission and the energy status of each record, and the Sustainability Dashboard shows the sustainability team where consumption is highest.

Automation is a central feature of low-code platforms. Rules, triggers and scheduled actions can perform repetitive tasks without manual effort, such as blocking an abnormal reading or pushing a new complaint to the CRM. Because much of the routine programming is handled by the platform, development effort is reduced and applications can be delivered faster. Low-code platforms are also accessible to a wider group of people: professional developers use them to speed up their work, and business users with limited programming experience can take part in building simple applications.

However, low-code does not remove the need for technical understanding. Concepts such as data models, logic, integration and security remain important, and developers who understand programming and technical principles are better able to design applications, extend them where the platform falls short and resolve problems when they occur.

Career Pathways in Low-Code Development

A range of professional roles is associated with low-code development. They differ in the depth of technical knowledge and the scope of responsibility. The major roles are described below, together with an example of how each role could relate to the Smart Campus application.

Role	Focus	Typical Responsibilities	Example in this project
Low-Code Application Developer	Building applications on low-code platforms	Creating forms, reports and workflows; configuring application logic and automation.	Builds the Energy, Water, Waste and Complaint forms and the consumption reports.

Role	Focus	Typical Responsibilities	Example in this project
Business Application Developer	Applications for specific business processes	Converting business requirements into working applications.	Turns the campus's monitoring, alerting and complaint-handling process into the application.
Automation Developer	Process automation	Designing automated workflows; reducing repetitive manual processes using triggers, workflows and automation tools.	Calculates CO_2 and the energy status automatically and blocks abnormal readings.
Integration Developer	Connecting systems	Working with APIs and external services; connecting applications and systems; handling data exchange between platforms.	Connects the complaint form to Zoho CRM and the AI assistant to an external AI service.
Low-Code Consultant	Advising on low-code solutions	Understanding business requirements; recommending suitable low-code solutions; helping organisations adopt low-code technologies.	Advises a campus on whether and how the tracker fits its sustainability programme.
Solution Developer / Platform Specialist	Advanced applications	Building more advanced applications; handling customisation and platform capabilities; working with integrations and complex requirements.	Writes custom Deluge functions, such as getTotalEnergy in Experiment 14.
Solution Architect	Solution design	Designing larger application solutions; planning architecture, integration and scalability; connecting business requirements with technical implementation.	Plans the data model, user roles and scaling for a multi-campus sustainability network.
Technical Lead / Enterprise Application Architect	Enterprise-level guidance	Guiding enterprise-level application development; handling technical decisions, governance, scalability and integration.	Sets standards, security and governance for the application across an education group.

Figure 1 shows how these roles form a staircase. The first step is the Foundation (basic programming, database concepts, UI/UX fundamentals, and workflow and automation). The next steps are the Low-Code Developer (application development, forms and reports, and business workflows), the Specialized Developer (automation, integration or business applications) and the Advanced Developer / Consultant (including Solution Developer and Platform Specialist roles). The top steps are the Solution Architect and the Technical Lead / Enterprise Architect. The strip at the bottom of the figure lists the core skills that support every step.

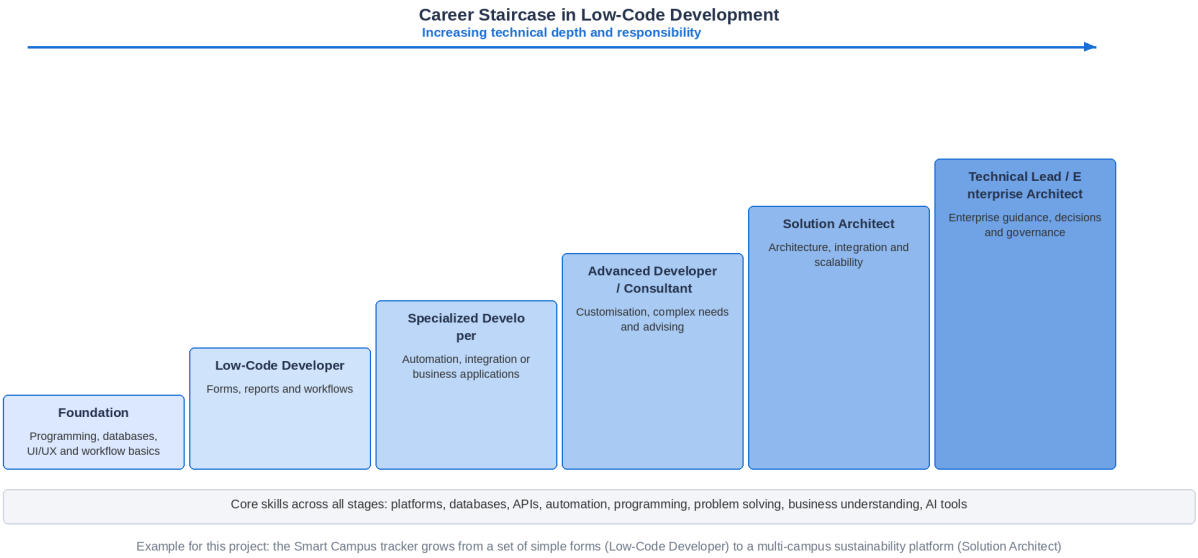


Figure 1: Career staircase and professional roles in low-code development.

Skills Required for Low-Code Careers

Low-code careers require a combination of platform knowledge and wider technical and professional skills.

Skill	Relevance
Low-Code Platform Knowledge	Understanding the tools, components and limits of a platform in order to build applications efficiently.
Database Concepts	Designing tables, fields and relationships so that application data is organised correctly.
Workflow and Automation	Modelling business processes and automating repetitive tasks using rules and triggers.
APIs and Integrations	Connecting applications with other systems and exchanging data between them.
Basic Programming	Writing scripts and custom logic where standard components are not sufficient.
UI/UX Fundamentals	Designing screens that are clear, consistent and easy for users to work with.
Problem Solving	Analysing requirements and difficulties and choosing suitable solutions.
Business Process Understanding	Understanding how an organisation works so that applications support real needs.
Data Handling	Managing, validating, processing and reporting on data.
AI-Assisted Development	Using AI tools to assist in building, testing and improving applications.
Communication and Requirement Analysis	Gathering requirements from users and explaining solutions to technical and non-technical people.

Low-code development does not mean that technical knowledge becomes unnecessary. Developers increasingly benefit from combining platform knowledge with programming, databases, APIs, automation and business understanding.

Skills Applied in the Smart Campus Project

Building this project draws on several of the skills listed above. The table links each skill to the part of the application where it is used.

Skill	Where it is used in the project
Platform knowledge	Creating the forms, reports, pages and workflows of the application in Zoho Creator.
Database concepts	Relating buildings to energy, water and waste records through lookup fields.
Workflow and automation	Calculating CO_2, setting statuses and pushing complaints to the CRM automatically.
Basic programming	Writing Deluge scripts and custom functions for logic the standard components do not cover.
UI/UX fundamentals	Keeping the data-entry forms and the dashboard short and clear for staff and students.
Data handling	Validating readings, aggregating totals and reporting on consumption and recycling.
Requirement analysis	Understanding what facility staff, students and sustainability officers need from the system.

Career Progression

A realistic progression in low-code development is shown below. Responsibilities and technical depth generally increase at each stage.

Stage	Description
Foundation	Basic programming, database concepts, UI/UX fundamentals and an understanding of workflow and automation.
Low-Code Developer	Building applications with forms, reports and workflows on a low-code platform.
Specialized Developer	Focusing on an area such as automation, integration or business applications.
Advanced Developer / Consultant	Handling complex requirements, customisation and advising organisations on solutions.
Solution Architect	Designing larger solutions, including architecture, integration and scalability.
Technical Lead / Enterprise Architect	Guiding enterprise-level development, technical decisions and governance.

Beginners can start with platform fundamentals, forms, reports, workflows and basic databases, as in the first version of the campus tracker. With experience, they can progress towards APIs, integrations, custom functions, advanced automation, architecture and enterprise solutions. As a person progresses, the work moves from building individual application components to designing complete solutions and guiding technical decisions.

Future of Low-Code Development

Low-code development is expected to become increasingly integrated with artificial intelligence, automation, cloud services and enterprise systems. The major trends are as follows:

- **AI-assisted application development:** AI tools that help in creating and improving applications.
- **Intelligent workflow automation:** automation that is supported by intelligent features.

- **Natural-language application building:** describing an application in everyday language to help build it.
- **API and cloud integration:** closer connection with cloud services and other systems through APIs.
- **Citizen development:** wider participation by business users in building applications.
- **Enterprise adoption:** use of low-code in larger organisations and for larger applications.
- **Low-code and pro-code collaboration:** combined use of visual development and custom code.
- **AI-powered testing and optimization:** AI support in testing applications and improving their performance.
- **Faster application delivery:** shorter time from requirement to working application.
- **Evolving developer roles:** changes in the work that developers do.

Figure 2 places these ten trends around a central hub, Future of Low-Code Development. The statement at the foot of the figure summarises the outlook: low-code will not eliminate developers but will change how they build, automate, integrate and solve business problems. For a project such as the Smart Campus tracker, these trends could mean describing a new sustainability form in plain language, flagging abnormal consumption automatically, forecasting next month's energy use, or connecting the application to cloud-based meter and messaging services.

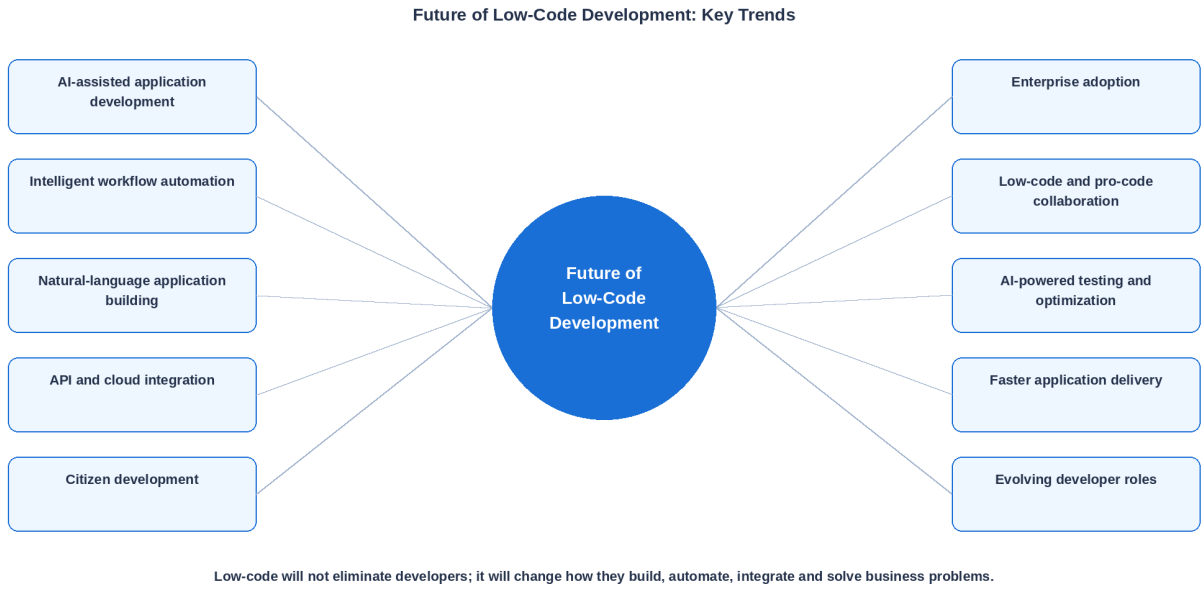


Figure 2: Major trends shaping the future of low-code development.

Low-Code + AI + Pro-Code

Low-code, AI and pro-code complement one another rather than competing. Each contributes different strengths to application development.

Low-Code	AI	Pro-Code
Faster application development	Intelligent automation	Custom business logic
Visual development	Natural-language interaction	Advanced processing
Pre-built components	Application assistance	APIs and integrations
Workflow automation	Data analysis and insights	Greater control

Low-Code	AI	Pro-Code
Accessibility	Testing and optimization	Specialized functionality

Low-code provides the foundation of the application, AI adds intelligence and efficiency, and pro-code enables customisation. In the campus project, the forms and workflows come from low-code, an anomaly-flagging or forecasting helper could come from AI, and special calculations such as region-specific emission factors, or integrations, are written as Deluge functions. Figure 3 presents the three as pillars that together support smarter applications for a digital future, with the expected results of faster development, higher productivity, better customisation, smarter decision making, improved collaboration and long-term scalability. Future developers may therefore need to combine all three rather than rely on only one approach.

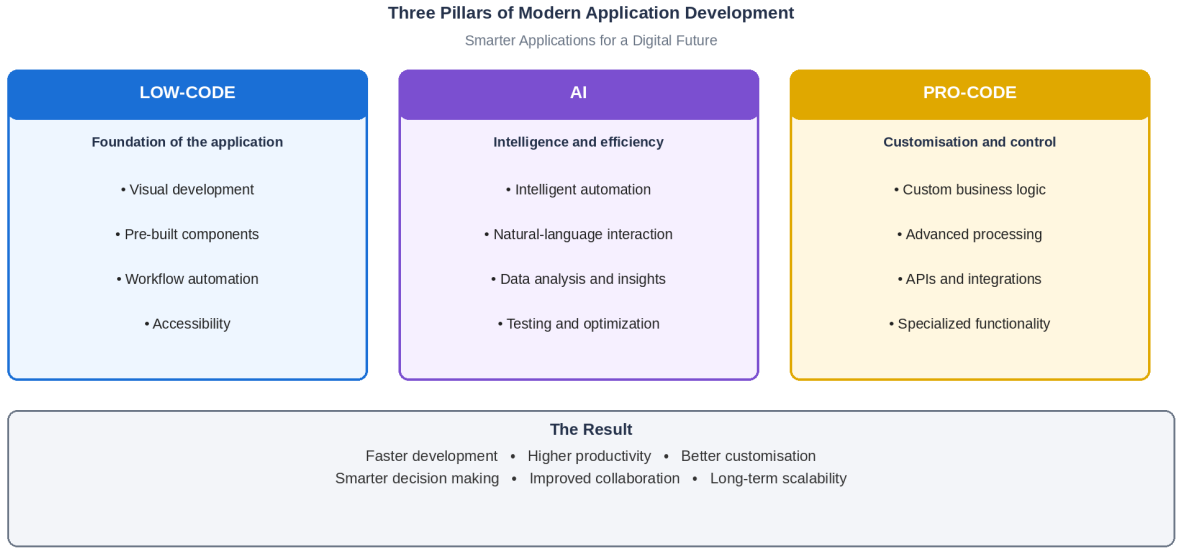


Figure 3: Low-code, AI and pro-code as the pillars of modern application development.

Future Developer Skill Set

The future developer is expected to have a combination of the following:

Low-Code Platforms + Programming Fundamentals + APIs & Integration + Databases + AI Tools + Business Understanding

Component	Role
Low-Code Platforms	Building applications quickly using visual tools, components and workflows.
Programming Fundamentals	Writing custom logic, understanding how applications work and troubleshooting problems.
APIs & Integration	Connecting applications with other systems and services.
Databases	Organising and managing application data.
AI Tools	Assisting with development, automation, testing and data analysis.
Business Understanding	Ensuring that applications solve real business needs.

Low-code platforms can reduce repetitive development effort, while programming knowledge and technical understanding remain valuable for customisation, troubleshooting, integration and advanced solutions.

Advantages of Low-Code Development

- **Faster development:** visual tools and ready components shorten development time.
- **Reduced repetitive coding:** the platform handles much of the routine programming.
- **Faster prototyping:** working versions of an application can be produced early and refined, for example a first version of the energy form.
- **Workflow automation:** repetitive business processes can be automated.
- **Easier application maintenance:** visual configuration can make applications easier to understand and modify.
- **Improved collaboration:** technical and business teams can work together more easily on the same application.
- **Wider access to application development:** people with limited programming experience can contribute.
- **Faster response to business requirements:** applications can be changed quickly as needs change, such as adding a new complaint category.

Challenges of Low-Code Development

- **Platform limitations:** a platform may not support every requirement.
- **Vendor dependency:** the application depends on the capabilities, pricing and restrictions of the chosen platform.
- **Complex customization requirements:** highly specialised needs may exceed standard components.
- **Integration challenges:** connecting with some systems, especially legacy systems, can be difficult.
- **Scalability considerations:** very large data volumes and user numbers need careful evaluation.
- **Security and governance:** access control, compliance and management of many applications need attention, for instance to protect the personal details in complaints.
- **Need for technical knowledge:** advanced applications still require technical understanding.
- **Requirement for pro-code extensions:** complex scenarios may need custom code in addition to low-code.

These challenges do not make low-code unsuitable. They indicate that the approach should be chosen according to the requirements of the application.

Career Opportunities

Low-code professionals can work in organisations and areas such as the following:

- Enterprise application development
- Business process automation
- Digital transformation
- IT consulting
- Workflow automation
- System integration
- Cloud application development
- Internal business tools

- Data-driven applications

Because low-code platforms are used to automate and digitise business processes, career opportunities can exist across different industries, including education and sustainability management, where applications like the one in this project help institutions track resources and reduce their carbon footprint.

Future Career Outlook

Low-code development is likely to evolve rather than simply replace conventional software development. Several areas are expected to grow in importance: AI-assisted development, hybrid low-code and pro-code development, integration skills, cloud technologies, data management, business understanding and solution architecture.

As platforms and AI tools take over more repetitive implementation, future developers may spend less time on routine work and more time on problem solving, architecture, customisation, integration and business outcomes.

Conclusion

Low-code development provides an accessible and efficient approach to application development, and it is creating a growing range of career opportunities. Visual tools, reusable components and automation allow applications to be delivered faster and allow both professional developers and business users to contribute. Career progression can begin with low-code application development, such as building the forms and workflows of the Smart Campus Sustainability & Carbon Tracker, and advance towards automation, integration, consulting, solution development and architecture. At each stage, technical depth and responsibility increase, and skills in databases, APIs, programming, automation and business understanding become more important.

The future of low-code development will be strongly influenced by AI, automation, APIs, cloud technologies and collaboration between low-code and pro-code approaches. Developer roles are expected to evolve rather than disappear. A strong future-ready developer should therefore combine low-code platform knowledge with programming fundamentals, databases, APIs, AI tools and business understanding.